TALLER PARCIAL - LABORATORIO SISTEMAS DISTRIBUIDOS

UNIVERSIDAD TECNOLÓGICA DE PEREIRA

Ingeniería de Sistemas y Computación

Octubre de 2026

1. Propósito del laboratorio

Diseñar, planear, implementar y demostrar un Sistema Distribuido (SD) cuyos componentes se ejecuten en máquinas independientes y se comuniquen mediante una red. El reto no consiste únicamente en ejecutar varios programas, sino en lograr que hosts diferentes colaboren para prestar un servicio que pueda percibirse como un solo sistema.

2. Reto general

Cada grupo deberá proponer y construir un pequeño Sistema Distribuido. Antes de programar deberá definir el problema, la arquitectura, los nodos, el mecanismo de comunicación, la estrategia de distribución de la información, la replicación y el comportamiento esperado ante fallos.

Ejemplos de servicios posibles:

- Almacenamiento distribuido de archivos o documentos.
- Repositorio distribuido de imágenes o registros.
- Sistema distribuido de mensajes.
- Procesamiento distribuido de información.
- Otra propuesta técnicamente justificable y aprobada por el docente.

3. Restricción fundamental: no es un ejercicio de localhost

NO se aceptará como solución completa una arquitectura ejecutada únicamente sobre una sola máquina física. Cuatro terminales, cuatro procesos, cuatro contenedores Docker o cuatro máquinas virtuales alojadas exclusivamente en el mismo computador no satisfacen el objetivo central del laboratorio.

```
NO VÁLIDO

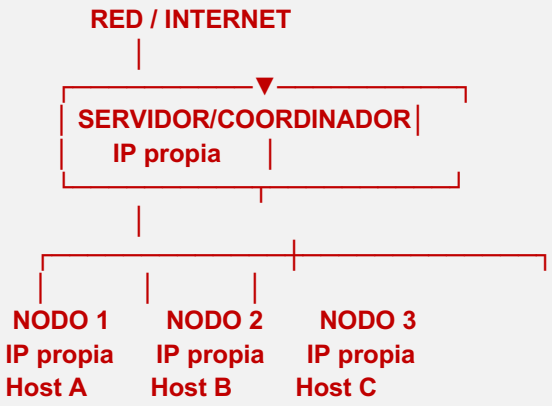
UN SOLO COMPUTADOR
  localhost
    ├─ Coordinador
```

```
├─ Nodo 1
├─ Nodo 2
└─ Nodo 3
```

Docker y las máquinas virtuales sí pueden utilizarse como tecnologías de despliegue, siempre que los componentes evaluados estén distribuidos entre hosts independientes y exista comunicación real por red.

4. Arquitectura mínima esperada

La referencia recomendada es un coordinador/servidor y tres nodos desplegados en máquinas independientes. Cada host deberá poseer una dirección IP distinta y ser alcanzable desde los demás componentes del sistema.



Los hosts pueden estar en el mismo salón, en diferentes redes domésticas o incluso en lugares diferentes. Lo importante es que sean máquinas independientes y que la aplicación se comunique mediante la red.

5. Herramientas permitidas

El grupo tiene libertad tecnológica, siempre que pueda justificar sus decisiones y demostrar el funcionamiento.

Componente	Alternativas
Sistemas operativos	Linux, Windows, macOS o combinación
Lenguajes	Python, Java, JavaScript/Node.js, C/C++ u otro justificado
Comunicación	Sockets TCP, HTTP/REST, RPC, mensajería u otra alternativa
Red entre hosts	LAN, Tailscale, ZeroTier, VPN o tecnología equivalente
Despliegue	Ejecución directa, Docker o máquinas virtuales distribuidas entre hosts

6. Conexión de máquinas ubicadas en lugares diferentes

Si los integrantes no están en la misma red local, pueden utilizar una red privada virtual. Tailscale o ZeroTier son alternativas que permiten que computadores conectados desde redes físicas distintas se comuniquen mediante direcciones IP de una red privada.



7. Fase de diseño y planeación

Antes de escribir código, el grupo deberá documentar las siguientes decisiones:

- Propósito: ¿qué problema resuelve y qué servicio ofrece?
- Arquitectura: ¿qué nodos existen y qué responsabilidad tiene cada uno?
- Comunicación: ¿qué protocolo o mecanismo utilizarán?
- Datos: ¿qué información viajará por la red y qué se almacenará?
- Distribución: ¿cómo decide el sistema qué nodo utilizar?
- Replicación: ¿qué información existirá en más de un nodo?
- Fallos: ¿qué debe suceder cuando un nodo deja de responder?

8. Inventario de infraestructura

Antes de implementar el servicio, complete y presente la siguiente tabla:

Host	Responsable	Rol	IP	Puerto	Estado
A		Coordinador			
B		Nodo 1			
C		Nodo 2			
D		Nodo 3			

9. Prueba de conectividad

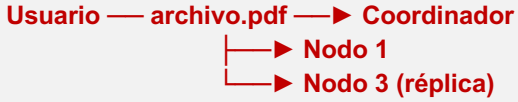
El grupo deberá demostrar que los hosts se alcanzan entre sí. Puede utilizar, entre otras, pruebas como:

```
ping IP_NODO_1
ping IP_NODO_2
ping IP_NODO_3
```

Importante: que exista respuesta a ping demuestra conectividad, pero no demuestra por sí solo que exista un Sistema Distribuido. Después deberá demostrarse comunicación entre procesos y prestación del servicio.

10. El usuario debe percibir un solo servicio

El usuario no debería necesitar conocer en qué nodo se encuentra físicamente la información. Por ejemplo, en un sistema distribuido de archivos, el usuario solicita SUBIR o DESCARGAR un recurso y la arquitectura decide qué nodos utilizar.

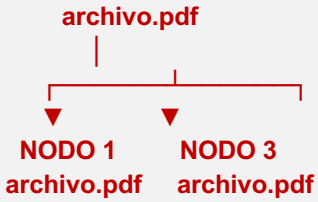


11. La información debe existir realmente en los nodos

No será suficiente mostrar un mensaje que diga “archivo replicado”. Si el sistema afirma que un recurso está almacenado en Nodo 1 y Nodo 3, deberá ser posible verificar en ambos hosts la existencia real del archivo o de la información correspondiente.

12. Distribución y replicación

El sistema deberá demostrar que utiliza varios nodos. Además, al menos un recurso deberá existir en más de un nodo para permitir una prueba de disponibilidad ante fallo.



13. Prueba obligatoria de fallo

Durante la demostración, uno de los nodos que contiene una réplica será detenido. El sistema deberá evidenciar qué ocurre y, cuando exista otra réplica disponible, permitir recuperar el recurso desde otro nodo.



La evidencia esperada no es únicamente que el segundo archivo exista: la aplicación deberá poder utilizar la réplica para continuar ofreciendo el recurso cuando la estrategia implementada así lo permita.

14. Fallo del coordinador: análisis

Como ejercicio de análisis, el grupo deberá detener el coordinador manteniendo activos los nodos. Deberá explicar qué operaciones continúan siendo posibles, cuáles dejan de funcionar y si el coordinador constituye un punto único de fallo (Single Point of Failure). No es obligatorio resolver completamente este problema en la primera versión, pero sí comprenderlo y proponer una alternativa.

15. Evidencias de entrega

- Diagrama de arquitectura con hosts, nodos, IP, puertos, protocolo y responsabilidades.
- Descripción del problema y del servicio distribuido.
- Tabla de infraestructura.
- Código fuente organizado por componentes.
- Evidencia de conectividad entre hosts.
- Evidencia de comunicación entre procesos.
- Evidencia de información almacenada físicamente en los nodos.
- Evidencia de distribución y replicación.
- Evidencia de caída de un nodo y recuperación desde otra réplica.
- Análisis técnico de decisiones, resultados, ventajas, limitaciones y fallos.

16. Secuencia mínima que debe poder demostrarse

MÁQUINAS DIFERENTES
↓
IP DIFERENTES
↓
COMUNICACIÓN POR RED
↓
SERVICIO DISTRIBUIDO
↓
INFORMACIÓN EN NODOS REALES
↓
REPLICACIÓN
↓
CAÍDA DE UN NODO
↓
RECUPERACIÓN DESDE OTRO NODO

17. Pregunta central para la sustentación

¿Por qué lo que construimos puede considerarse un Sistema Distribuido y no simplemente varios programas ejecutándose en computadores diferentes?

La respuesta deberá sustentarse utilizando la arquitectura, la comunicación real por red, la ubicación de la información, la replicación y el comportamiento observado ante fallos.

18. Criterios que debe tenerse presente para comprobar y evaluar en vivo su laboratorio

- Solicitar la IP de cada host y comprobar su conectividad.
- Solicitar que se muestre dónde se encuentra físicamente un archivo o dato.
- Detener aleatoriamente uno de los nodos.
- Solicitar nuevamente el recurso y observar qué componente responde.
- Detener el coordinador y analizar el comportamiento.
- Preguntar qué cambiaría si el sistema pasara de tres a cien nodos.